

IF 260

Sistem Operasi

Minggu 6

Dr. Ananda Kusuma
e-mail: ananda.kusuma@lecturer.umn.ac.id

Universitas Multimedia Nusantara
Serpong, Tangerang

Agenda

- **Review**
- **Topik Minggu 6:**
 - **CPU Scheduling (Penjadwalan CPU)**
 - Batch
 - Interactive
 - Real-Time

Review:

Dining Philosophers:

Solusi dengan semaphores (1)

```
#define N          5                /* number of philosophers */
#define LEFT      (i+N-1)%N        /* number of i's left neighbor */
#define RIGHT     (i+1)%N          /* number of i's right neighbor */
#define THINKING  0                /* philosopher is thinking */
#define HUNGRY    1                /* philosopher is trying to get forks */
#define EATING    2                /* philosopher is eating */
typedef int semaphore;             /* semaphores are a special kind of int */
int state[N];                     /* array to keep track of everyone's state */
semaphore mutex = 1;              /* mutual exclusion for critical regions */
semaphore s[N];                   /* one semaphore per philosopher */

void philosopher(int i)           /* i: philosopher number, from 0 to N-1 */
{
    while (TRUE) {                /* repeat forever */
        think();                  /* philosopher is thinking */
        take_forks(i);            /* acquire two forks or block */
        eat();                    /* yum-yum, spaghetti */
        put_forks(i);             /* put both forks back on table */
    }
}
```

Dining Philosophers:

Solusi dengan semaphores (2)

...

```
void take_forks(int i)                                /* i: philosopher number, from 0 to N-1 */
{
    down(&mutex);                                       /* enter critical region */
    state[i] = HUNGRY;                                  /* record fact that philosopher i is hungry */
    test(i);                                           /* try to acquire 2 forks */
    up(&mutex);                                         /* exit critical region */
    down(&s[i]);                                        /* block if forks were not acquired */
}
```

...

Dining Philosophers:

Solusi dengan semaphores (3)

...

```
void put_forks(i)                /* i: philosopher number, from 0 to N-1 */
{
    down(&mutex);                /* enter critical region */
    state[i] = THINKING;         /* philosopher has finished eating */
    test(LEFT);                  /* see if left neighbor can now eat */
    test(RIGHT);                 /* see if right neighbor can now eat */
    up(&mutex);                  /* exit critical region */
}
```

```
void test(i) /* i: philosopher number, from 0 to N-1 */
{
    if (state[i] == HUNGRY && state[LEFT] != EATING && state[RIGHT] != EATING) {
        state[i] = EATING;
        up(&s[i]);
    }
}
```

Mekanisme Sinkronisasi Lainnya:

- Monitor**

Monitors

- Higher level synchronization (language concept)
- Kumpulan procedure, variable, dan struktur data yang dikelompokkan menjadi satu modul atau package
- Process dapat memanggil procedures yang ada di dalam monitor, tapi tidak dapat mengakses struktur data dari monitor
- Hanya ada satu process yang aktif di dalam monitor pada satu waktu
 - Compiler membuat call ke monitor procedures berbeda dengan procedures biasa → ada beberapa instruksi ditambahkan untuk memeriksa apakah ada process lain yang sedang aktif di dalam monitor
 - Pada Java, penggunaan monitor dengan synchronized method/statement

Producer-consumer problem menggunakan monitor

```
procedure producer;  
begin  
    while true do  
    begin  
        item = produce_item;  
        ProducerConsumer.insert(item)  
    end  
end;  
procedure consumer;  
begin  
    while true do  
    begin  
        item = ProducerConsumer.remove;  
        consume_item(item)  
    end  
end;
```

```
monitor ProducerConsumer  
    condition full, empty;  
    integer count;  
    procedure insert(item: integer);  
    begin  
        if count = N then wait(full);  
        insert_item(item);  
        count := count + 1;  
        if count = 1 then signal(empty)  
    end;  
    function remove: integer;  
    begin  
        if count = 0 then wait(empty);  
        remove = remove_item;  
        count := count - 1;  
        if count = N - 1 then signal(full)  
    end;  
    count := 0;  
end monitor;
```

Contoh implementasi monitor dalam Java

```
public class ProducerConsumer {
    static final int N = 100;    // constant giving the buffer size
    static producer p = new producer();    // instantiate a new producer thread
    static consumer c = new consumer();    // instantiate a new consumer thread
    static our_monitor mon = new our_monitor();    // instantiate a new monitor

    public static void main(String args[]) {
        p.start();    // start the producer thread
        c.start();    // start the consumer thread
    }

    static class producer extends Thread {
        public void run() { // run method contains the thread code
            int item;
            while (true) {    // producer loop
                item = produce_item();
                mon.insert(item);
            }
        }
        private int produce_item() { ... }    // actually produce
    }

    static class consumer extends Thread {
        public void run() { run method contains the thread code
            int item;
            while (true) {    // consumer loop
                item = mon.remove();
                consume_item (item);
            }
        }
        private void consume_item(int item) { ... } // actually consume
    }
}
```

Contoh implementasi monitor dalam Java

```
static class our_monitor { // this is a monitor
    private int buffer[] = new int[N];
    private int count = 0, lo = 0, hi = 0; // counters and indices

    public synchronized void insert(int val) {
        if (count == N) go_to_sleep(); // if the buffer is full, go to sleep
        buffer[hi] = val; // insert an item into the buffer
        hi = (hi + 1) % N; // slot to place next item in
        count = count + 1; // one more item in the buffer now
        if (count == 1) notify(); // if consumer was sleeping, wake it up
    }

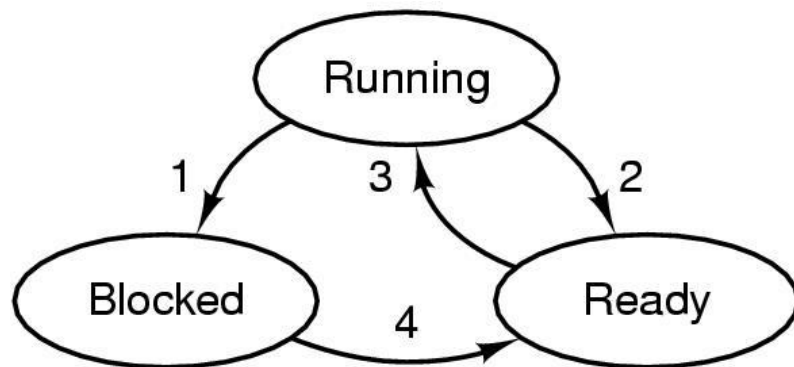
    public synchronized int remove() {
        int val;
        if (count == 0) go_to_sleep(); // if the buffer is empty, go to sleep
        val = buffer[lo]; // fetch an item from the buffer
        lo = (lo + 1) % N; // slot to fetch next item from
        count = count - 1; // one few items in the buffer
        if (count == N - 1) notify(); // if producer was sleeping, wake it up
        return val;
    }
    private void go_to_sleep() { try{wait();} catch(InterruptedException exc) {};}
}
```

CPU Scheduling (Penjadwalan CPU)

Levels of Scheduling

(Klasifikasi Tingkatan Penjadwalan)

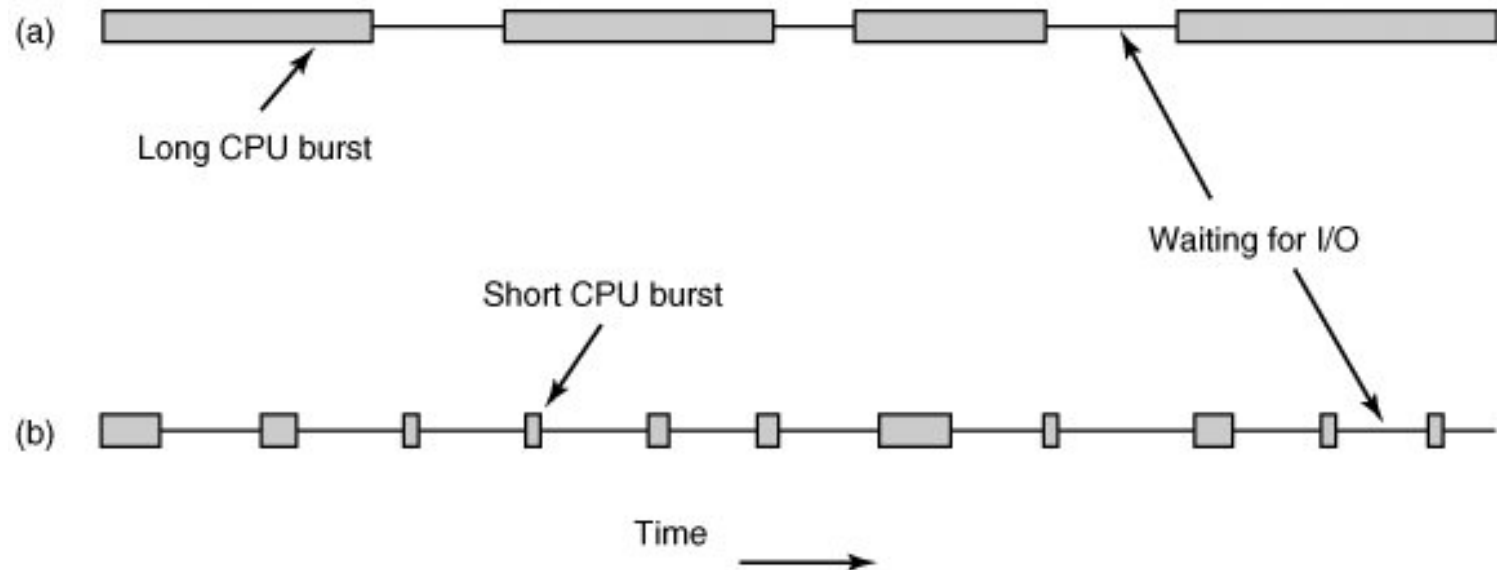
- **Long-term scheduling (job scheduling)**
 - Pemilihan jobs yang akan di-load untuk berkompetisi mendapatkan system resources
- **Medium-term scheduling**
 - Pemilihan jobs untuk di-suspend/resume dalam rangka menyeimbangkan beban system
- **Short-term scheduling (CPU scheduling) dan Dispatching**



1. Process blocks for input
2. Scheduler picks another process
3. Scheduler picks this process
4. Input becomes available

Issues on Scheduling (Hal-hal terkait Penjadwalan)

- **Process behaviour: cpu-bound atau I/O-bound**



- **How to deal with clock interrupts:**
 - Preemptive → process dapat di-interrupt dan melepaskan jatah CPU nya
 - non-preemptive → begitu jatah CPU diberikan, process dapat menggunakannya terus sampai process exits atau switches to waiting state

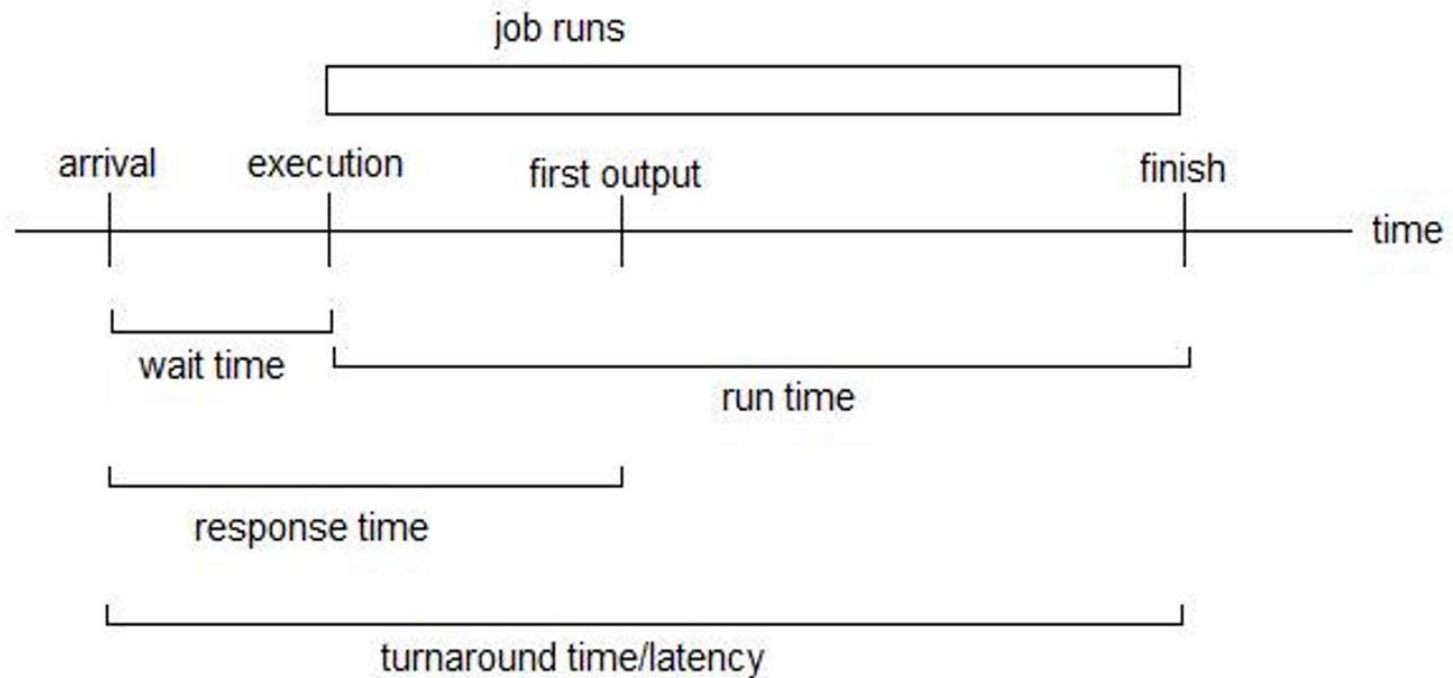
Berdasarkan Kategori Aplikasi

- **Batch**
 - First-Come First-Served (FCFS)
 - Shortest Job First
 - Shortest Remaining Time Next
- **Interactive**
 - Round-Robin
 - Priority
- **Real-time**
 - Rate monotonic
 - Earliest Deadline First

Scheduling Objectives

- **Untuk semua system**
 - Fairness
 - Policy enforcement
 - Balance
- **Batch**
 - Throughput
 - Turnaround time
 - CPU Utilization
- **Interactive**
 - Response time
 - Proportionality
- **Real-time**
 - Meeting deadline
 - Predictability

Scheduling Metrics



Batch Systems

First-Come First-Served (FCFS)

- **Alokasikan CPU ke process sesuai dengan urutan kedatangan**
- **Implementasi**
 - **Gunakan satu buah antrean FIFO (First In First Out)**
- **Performance metric: average waiting time atau average turnaround time**

Average waiting time = durasi waktu tunggu rata-rata

Waktu tunggu = durasi waktu dari process/job/task tiba sampai diambil oleh CPU untuk dikerjakan

Average turnaround time = durasi waktu rata-rata dari process/job/task tiba sampai selesai dikerjakan oleh CPU

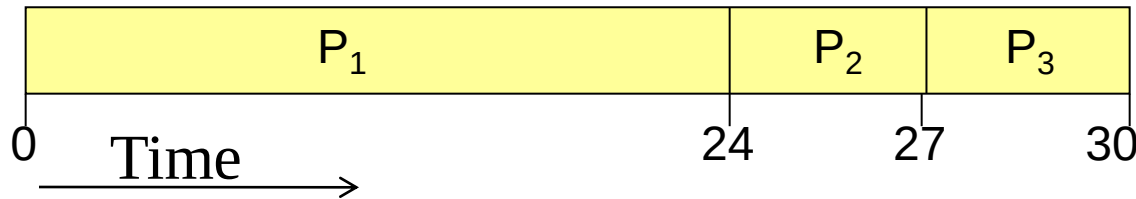
Arrival time = waktu kedatangan

CPU run time = waktu yang diperlukan oleh CPU untuk menyelesaikan pekerjaan dari process/job/task

First-Come First-Served (FCFS): Contoh

- Misal urutan kedatangan (arrival) process: P1, P2, P3
- Panjang CPU run time (burst):

Process	Run Time
P1	24
P2	3
P3	3



- Waiting time:

Process	Waiting time
P1	0
P2	24
P3	27

- Average waiting time = $(0+24+27)/3 = 17$
- Bagaimana kalau seandainya urutan penjadwalan diubah?
 - P2,P3,P1 → average waiting time = $(6+0+3)/3 = 3$

Shortest Job First (SJF)

- Diasumsikan cpu run time (burst) tiap job diketahui
- Jadwalkan process dengan run time terpendek
- Contoh di slide sebelumnya, urutan penjadwalan process = P2,P3,P1
- Contoh: 4 jobs A, B, C,D dengan run time a,b,c,d
 - Average waiting time: $(0+a+a+b+a+b+c)/4 = (3a+2b+c)/4$
 - Average turnaround time: $(4a+3b+2c+d)/4$

Pembobotan ke metric lebih besar pada process yang lebih dulu diambil sehingga untuk meminimalkan diambil process yang terpendek

Shortest Job First (SJF)

- SJF optimal apabila semua jobs tersedia secara serentak

- Contoh:

Process	Arrival Time	Run Time
A	0	2
B	0	4
C	3	1
D	3	1
E	3	1

- Menggunakan SJF, order yang optimal -> A,B,C,D,E dengan

- Average waiting time = $(0+2+(6-3)+(7-3)+(8-3))/5 = 2.8$

- Average turnaround time =

$$(2+6+(7-3)+(8-3)+(9-3))/5 = 4.6$$

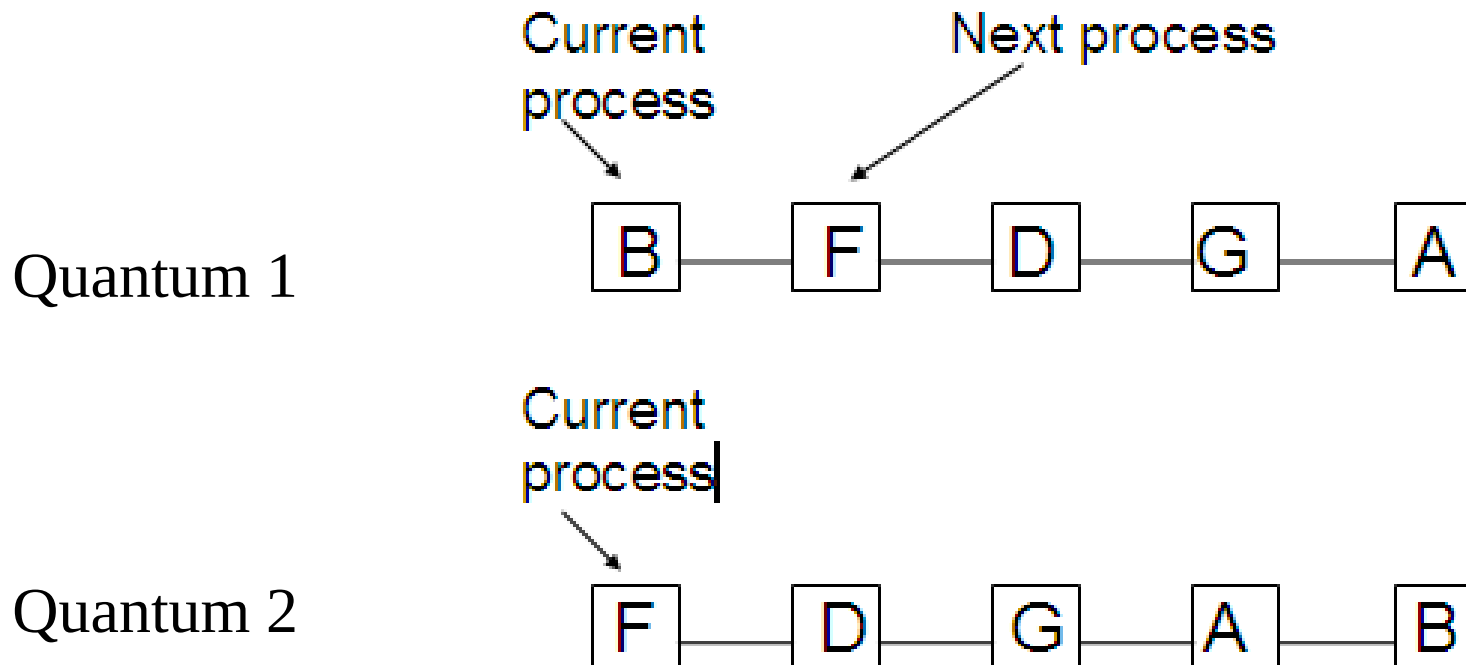
Shortest Time Remaining Next (STRN)

- Preemptive version dari SJF
- Scheduler menjadwalkan process dengan sis run time terpendek
- Jika ada job baru datang dengan run time yang lebih pendek dari process yang sedang berjalan, maka process yang sedang berjalan di-suspend dulu dan segera eksekusi process dari job yang baru.
- Menggunakan STRN:
 - Average turnaround time =
$$(2+9+(4-3)+(5-3)+(6-3))/5 = 3.4$$

Interactive Systems

Round-Robin Scheduling

- Penjadwalan CPU di mana tiap process mendapat giliran secara bergantian untuk menggunakan CPU. Interval waktu penggunaan CPU disebut *quantum*.
- Contoh:



Round-Robin: besaran quantum

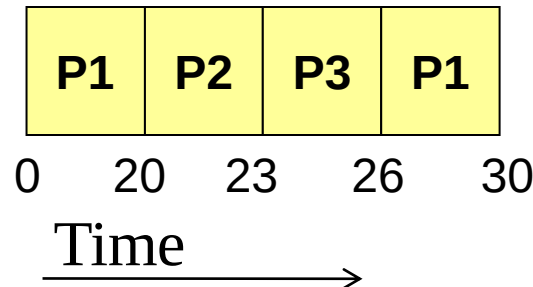
- Pertukaran penggunaan CPU dari satu process ke process berikutnya membutuhkan waktu untuk tugas administrasi (context switch), mis. saving dan loading registers, memory map, updating tabel, list, dll.
- Pemilihan besaran quantum perlu memperhatikan *trade-off*:
 - Terlalu kecil → terlalu banyak context switch dan sumber daya CPU terbuang untuk tugas administrasi
 - Terlalu besar → respons tidak bagus untuk layanan-layanan interactive singkat
- Contoh: waktu switching = 1 msec
 - Waktu quantum = 10 msec → $((1/11)*100\%) \cong 10\%$ → terlalu tinggi penggunaan CPU untuk keperluan administrasi
 - Waktu quantum = 100 msec → $(1/101)*100\%) \cong 1\%$ → utilisasi CPU bagus, tapi response time (interactive) jelek karena process terlalu lama menunggu giliran

Round-Robin: Contoh

- Misal: Time Quantum = 20, dan

Process	Run Time
P1	24
P2	3
P3	3

- Time line:



- Waiting Time:

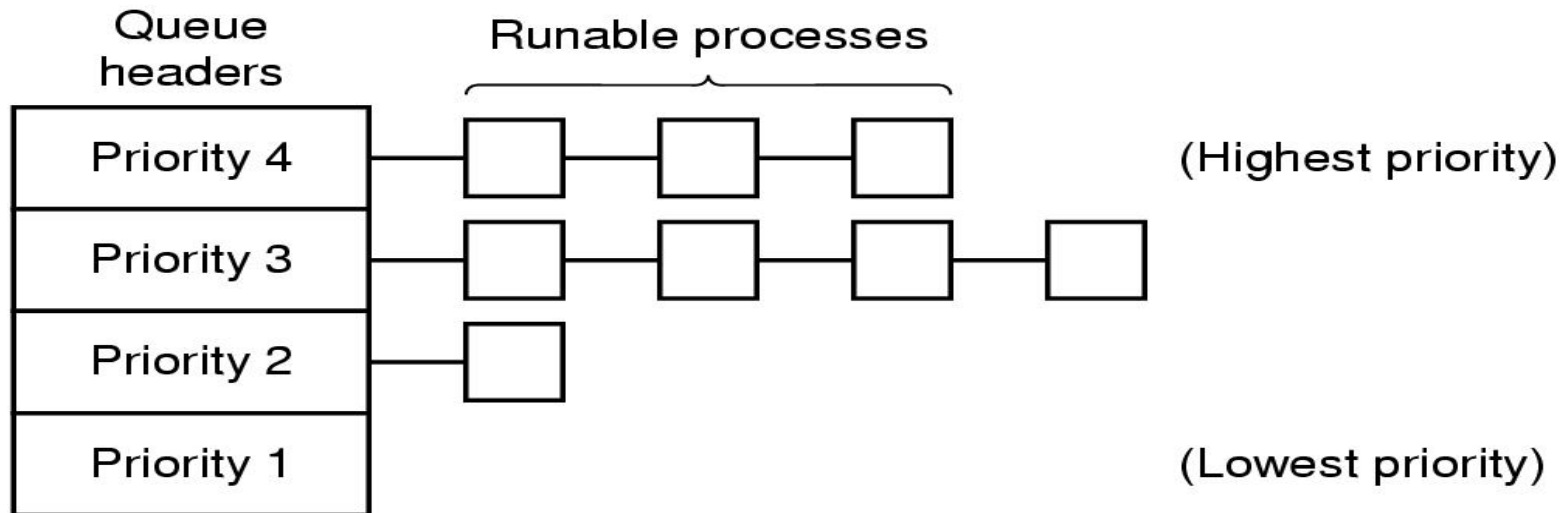
Process	Waiting time
P1	26
P2	20
P3	23

- Average waiting time = $(26+20+23)/3 = 23$
- Average turnaround time = $(30+23+26)/3 = 26.3$
 - Lebih jelek dibandingkan SJF, namun response time untuk tiap process lebih bagus karena ada kepastian giliran menggunakan CPU sesuai dengan time quantum yang ditetapkan

Priority Scheduling

(Penjadwalan dengan Prioritas)

- Tiap Process diberikan prioritas, dan prioritas yang lebih tinggi lebih dahulu dijalankan



- Untuk mencegah process dengan prioritas tertinggi berjalan tanpa batas, dapat digunakan clock interrupt untuk menurunkan prioritasnya sehingga process yang lain dapat berjalan

Priority Scheduling: Penentuan Prioritas

- Tiap Process diberikan prioritas, dan prioritas yang lebih tinggi lebih dahulu dijalankan
- Untuk mencegah process dengan prioritas tertinggi berjalan tanpa batas, dapat digunakan clock interrupt untuk menurunkan prioritasnya sehingga process yang lain dapat berjalan
- Cara sederhana:
 - Gunakan rumus $\text{Prioritas} = 1/f$, di mana f = prosentase waktu quantum yang digunakan
 - Contoh quantum = 50 msec
 - Process P1, baru running 1 msec, di-blocked, maka $f = 1/50 \rightarrow \text{Prioritas} = 50$
 - Process P2, baru running 25 msec, di-blocked, maka $f = 25/50 \rightarrow \text{Prioritas} = 2$
 - Process P1 lebih diprioritaskan dibandingkan P2 apabila keduanya ready menunggu penjadwalan CPU

Contoh Soal

Terdapat 5 jobs A, B, C ,D dan E dengan arrival time, run time dan priority masing-masing seperti tabel dibawah. (Catatan: untuk contoh soal ini, prioritas lebih tinggi ditunjukkan dengan nomor yang lebih tinggi). Tentukan average turnaround time dengan algoritma penjadwalan sbb.:

- First Come First Served
- Shortest Job First
- Shortest Remaining Time Next
- Non-preemptive Priority Scheduling

Job	Arrival Time (unit waktu)	Run Time (unit waktu)	Priority
A	0	7	1
B	1	9	4
C	2	2	1
D	5	5	3
E	7	3	2

Akhir Kuliah Minggu 6

Terima kasih atas perhatiannya!